

1.5 Control Flow

Control flow in Java is similar to that of other high-level languages. We review the basic structure and syntax of control flow in Java in this section, including method returns, **if** statements, **switch** statements, loops, and restricted forms of “jumps” (the **break** and **continue** statements).

1.5.1 The If and Switch Statements

In Java, conditionals work similarly to the way they work in other languages. They provide a way to make a decision and then execute one or more different statement blocks based on the outcome of that decision.

The If Statement

The syntax of a simple **if** statement is as follows:

```
if (booleanExpression)  
    trueBody  
else  
    falseBody
```

where *booleanExpression* is a boolean expression and *trueBody* and *falseBody* are each either a single statement or a block of statements enclosed in braces (“{” and “}”). Note that, unlike some similar languages, the value tested by an **if** statement in Java must be a boolean expression. In particular, it is definitely not an integer expression. Nevertheless, as in other similar languages, the **else** part (and its associated statement) in a Java **if** statement is optional. There is also a way to group a number of boolean tests, as follows:

```
if (firstBooleanExpression)  
    firstBody  
else if (secondBooleanExpression)  
    secondBody  
else  
    thirdBody
```

If the first boolean expression is false, the second boolean expression will be tested, and so on. An **if** statement can have an arbitrary number of **else if** parts. Braces can be used for any or all statement bodies to define their extent.

As a simple example, a robot controller might have the following logic:

```
if (door.isClosed())  
    door.open();  
    advance();
```

Notice that the final command, `advance()`, is not part of the conditional body; it will be executed unconditionally (although after opening a closed door).

We may nest one control structure within another, relying on explicit braces to mark the extent of the various bodies if needed. Revisiting our robot example, here is a more complex control that accounts for unlocking a closed door.

```
if (door.isClosed()) {  
    if (door.isLocked())  
        door.unlock();  
    door.open();  
}  
advance();
```

The logic expressed by this example can be diagrammed as a traditional *flowchart*, as portrayed in Figure 1.4.

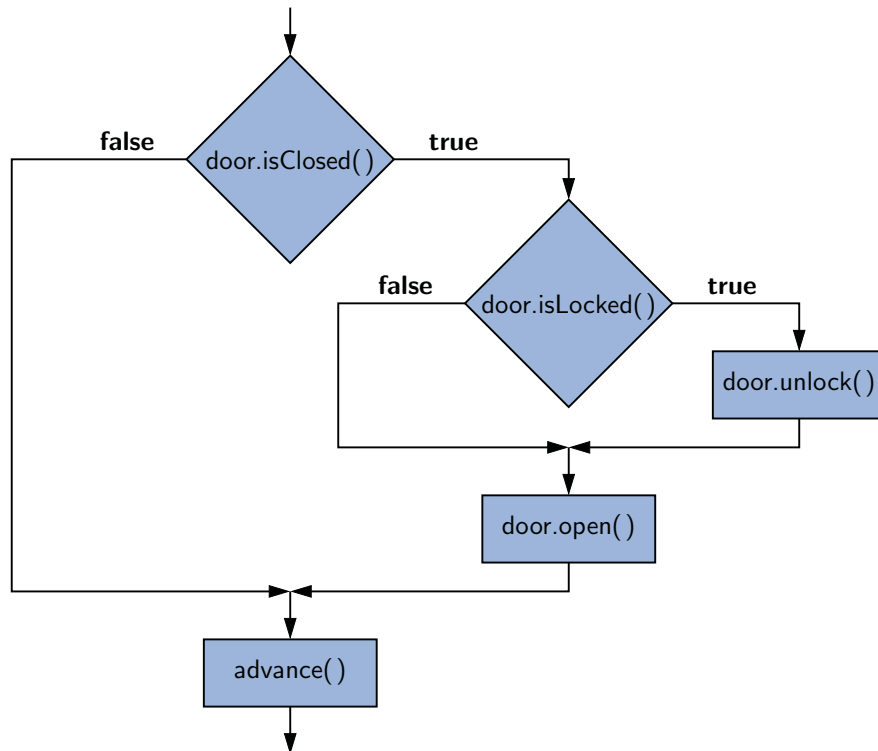


Figure 1.4: A flowchart describing the logic of nested conditional statements.

The following is an example of the nesting of **if** and **else** clauses.

```
if (snowLevel < 2) {  
    goToClass();  
    comeHome();  
} else if (snowLevel < 5) {  
    goSledding();  
    haveSnowballFight();  
} else  
    stayAtHome();           // single-statement body needs no { } braces
```

Switch Statements

Java provides for multiple-value control flow using the **switch** statement, which is especially useful with enum types. The following is an indicative example (based on a variable `d` of the `Day` enum type of Section 1.3).

```
switch (d) {  
    case MON:  
        System.out.println("This is tough.");  
        break;  
    case TUE:  
        System.out.println("This is getting better.");  
        break;  
    case WED:  
        System.out.println("Half way there.");  
        break;  
    case THU:  
        System.out.println("I can see the light.");  
        break;  
    case FRI:  
        System.out.println("Now we are talking.");  
        break;  
    default:  
        System.out.println("Day off!");  
}
```

The **switch** statement evaluates an integer, string, or enum expression and causes control flow to jump to the code location labeled with the value of this expression. If there is no matching label, then control flow jumps to the location labeled “**default**.” This is the only explicit jump performed by the **switch** statement, however, so flow of control “falls through” to the next case if the code for a case is not ended with a **break** statement (which causes control flow to jump to the end).

1.5.2 Loops

Another important control flow mechanism in a programming language is looping. Java provides for three types of loops.

While Loops

The simplest kind of loop in Java is a **while** loop. Such a loop tests that a certain condition is satisfied and will perform the body of the loop each time this condition is evaluated to be **true**. The syntax for such a conditional test before a loop body is executed is as follows:

```
while (booleanExpression)  
    loopBody
```

As with an **if** statement, *booleanExpression*, can be an arbitrary boolean expression, and the body of the loop can be an arbitrary block of code (including nested control structures). The execution of a **while** loop begins with a test of the boolean condition. If that condition evaluates to **true**, the body of the loop is performed. After each execution of the body, the loop condition is retested and if it evaluates to **true**, another iteration of the body is performed. If the condition evaluates to **false** when tested (assuming it ever does), the loop is exited and the flow of control continues just beyond the body of the loop.

As an example, here is a loop that advances an index through an array named *data* until finding an entry with value *target* or reaching the end of the array.

```
int j = 0;  
while ((j < data.length) && (data[j] != target))  
    j++;
```

When this loop terminates, variable *j*'s value will be the index of the leftmost occurrence of *target*, if found, or otherwise the length of the array (which is recognizable as an invalid index to indicate failure of the search). The correctness of the loop relies on the short-circuiting behavior of the logical **&&** operator, as described on page 25. We intentionally test *j* < *data.length* to ensure that *j* is a valid index, prior to accessing element *data[j]*. Had we written that compound condition with the opposite order, the evaluation of *data[j]* would eventually throw an *ArrayIndexOutOfBoundsException* if the target is not found. (See Section 2.4 for discussion of exceptions.)

We note that a **while** loop will execute its body zero times in the case that the initial condition fails. For example, our above loop will not increment the value of *j* if *data[0]* matches the target (or if the array has length 0).

Do-While Loops

Java has another form of the **while** loop that allows the boolean condition to be checked at the *end* of each pass of the loop rather than before each pass. This form is known as a **do-while** loop, and has syntax shown below:

```
do
    loopBody
while (booleanExpression)
```

A consequence of the **do-while** loop is that its body always executes at least once. (In contrast, a **while** loop will execute zero times if the initial condition fails.) This form is most useful for a situation in which the condition is ill-defined until after at least one pass. Consider, for example, that we want to prompt the user for input and then do something useful with that input. (We discuss Java input and output in more detail in Section 1.6.) A possible condition, in this case, for exiting the loop is when the user enters an empty string. However, even in this case, we may want to handle that input and inform the user that he or she has quit. The following example illustrates this case:

```
String input;
do {
    input = getInputString();
    handleInput(input);
} while (input.length() > 0);
```

For Loops

Another kind of loop is the **for** loop. Java supports two different styles of **for** loop. The first, which we will refer to as the “traditional” style, is patterned after a similar syntax as **for** loops in the C and C++ languages. The second style, which is known as the “for-each” loop, was introduced into Java in 2004 as part of the SE 5 release. This style provides a more succinct syntax for iterating through elements of an array or an appropriate container type.

The traditional **for**-loop syntax consists of four sections—an initialization, a boolean condition, an increment statement, and the body—although any of those can be empty. The structure is as follows:

```
for (initialization; booleanCondition; increment)
    loopBody
```

For example, the most common use of a **for** loop provides repetition based on an integer index, such as the following:

```
for (int j=0; j < n; j++)
    // do something
```

The behavior of a **for** loop is very similar to the following **while** loop equivalent:

```
{
    initialization;
    while (booleanCondition) {
        loopBody;
        increment;
    }
}
```

The *initialization* section will be executed once, before any other portion of the loop begins. Traditionally, it is used to either initialize existing variables, or to declare and initialize new variables. Note that any variables declared in the initialization section only exist in scope for the duration of the **for** loop.

The *booleanCondition* will be evaluated immediately before each potential iteration of the loop. It should be expressed similar to a **while**-loop condition, in that if it is **true**, the loop body is executed, and if **false**, the loop is exited and the program continues to the next statement beyond the **for**-loop body.

The *increment* section is executed immediately after each iteration of the formal loop body, and is traditionally used to update the value of the primary loop variable. However, the incrementing statement can be any legal statement, allowing significant flexibility in coding.

As a concrete example, here is a method that computes the sum of an array of **double** values using a **for** loop:

```
public static double sum(double[] data) {
    double total = 0;
    for (int j=0; j < data.length; j++)    // note the use of length
        total += data[j];
    return total;
}
```

As one further example, the following method computes the maximum value within a (nonempty) array.

```
public static double max(double[] data) {
    double currentMax = data[0];    // assume first is biggest (for now)
    for (int j=1; j < data.length; j++)    // consider all other entries
        if (data[j] > currentMax)    // if data[j] is biggest thus far...
            currentMax = data[j];    // record it as the current max
    return currentMax;
}
```

Notice that a conditional statement is nested within the body of the loop, and that no explicit “{” and “}” braces are needed for the loop body, as the entire conditional construct serves as a single statement.

For-Each Loop

Since looping through elements of a collection is such a common construct, Java provides a shorthand notation for such loops, called the *for-each loop*. The syntax for such a loop is as follows:

```
for (elementType name : container)  
    loopBody
```

where *container* is an array of the given *elementType* (or a collection that implements the `Iterable` interface, as we will later discuss in Section 7.4.1).

Revisiting a previous example, the traditional loop for computing the sum of elements in an array of **double** values can be written as:

```
public static double sum(double[ ] data) {  
    double total = 0;  
    for (double val : data)                // Java's for-each loop style  
        total += val;  
    return total;  
}
```

When using a for-each loop, there is no explicit use of array indices. The loop variable represents one particular element of the array. However, within the body of the loop, there is no designation as to which element it is.

It is also worth emphasizing that making an assignment to the loop variable has no effect on the underlying array. Therefore, the following method is an invalid attempt to scale all values of a numeric array.

```
public static void scaleBad(double[ ] data, double factor) {  
    for (double val : data)  
        val *= factor;                    // changes local variable only  
}
```

In order to overwrite the values in the cells of an array, we must make use of indices. Therefore, this task is best solved with a traditional **for** loop, such as the following:

```
public static void scaleGood(double[ ] data, double factor) {  
    for (int j=0; j < data.length; j++)  
        data[j] *= factor;                // overwrites cell of the array  
}
```

1.5.3 Explicit Control-Flow Statements

Java also provides statements that cause explicit change in the flow of control of a program.

Returning from a Method

If a Java method is declared with a return type of **void**, then flow of control returns when it reaches the last line of code in the method or when it encounters a **return** statement (with no argument). If a method is declared with a return type, however, the method must exit by returning an appropriate value as an argument to a **return** statement. It follows that the **return** statement *must* be the last statement executed in a method, as the rest of the code will never be reached.

Note that there is a significant difference between a statement being the last line of code that is *executed* in a method and the last line of code in the method itself. The following (correct) example illustrates returning from a method:

```
public double abs(double value) {  
    if (value < 0)           // value is negative,  
        return -value;      // so return its negation  
    return value;           // return the original nonnegative value  
}
```

In the example above, the line **return** `-value`; is clearly not the last line of code that is written in the method, but it may be the last line that is executed (if the original value is negative). Such a statement explicitly interrupts the flow of control in the method. There are two other such explicit control-flow statements, which are used in conjunction with loops and switch statements.

The break Statement

We first introduced use of the **break** command, in Section 1.5.1, to exit from the body of a switch statement. More generally, it can be used to “break” out of the innermost **switch**, **for**, **while**, or **do-while** statement body. When it is executed, a break statement causes the flow of control to jump to the next line after the loop or **switch** to the body containing the **break**.

The continue Statement

A **continue** statement can be used within a loop. It causes the execution to skip over the remaining steps of the *current iteration* of the loop body, but then, unlike the **break** statement, the flow of control returns to the top of the loop, assuming its condition remains satisfied.